Detecting and Preventing Data Poisoning Attacks

Project 10: AI354 Responsible AI

I23MA002 Devesh Singh Chauhan **U23EE102** Priyanshu Sanjay Gawas

Department of AI, SVNIT

April 23, 2026

Problem Statement

What is Data Poisoning?

Data poisoning is an adversarial attack where an attacker **injects malicious or manipulated samples into a model's training data**, causing the model to behave incorrectly at inference time.

The Core Threat

- Attacker controls part of training data
- Labels are **flipped** or features **perturbed**
- Backdoor triggers inserted silently
- Model **appears normal** until triggered

What We Need

- Statistical **detection** of suspicious samples
- **Robust** training algorithms
- Automated **alerting** pipelines
- Validation on **image** and **text** models

Key Challenge

Poisoned data often **looks perfectly normal** to a human reviewer; therefore, detection must be algorithmic and automated.

Motivation

Why does this matter?

- ML models are deployed in **safety-critical systems**: autonomous vehicles, medical diagnosis, fraud detection
- Training data is often sourced from **untrusted** third parties
- A 10% label-flip attack can drop accuracy by **6-9%**, which is enough to cause real harm
- **Supply-chain attacks** on AI are rising

Same image, different label: the attacker's trick

Real-World Incidents

- Microsoft Tay chatbot (2016): poisoned via user input
- Trojan attacks on ImageNet classifiers (2018)

Types of Data Poisoning Attacks

Label Flipping (Implemented)

- Attacker changes the **ground-truth labels** of training samples.
- E.g., Digit "8" labelled as "4".
- Simple but highly effective at scale.

Backdoor / Trojan Attack

- A hidden **trigger pattern** causes misclassification only when present at inference.

Clean-Label Poisoning

- Labels are kept **correct** but feature vectors are perturbed. Much harder to detect.

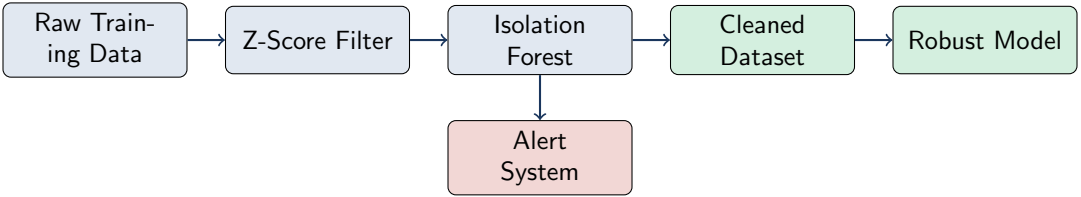
Feature Collision

- Poisoned samples are crafted to be **close in feature space** to a target class.

Our Focus

Label-flipping attack: most common, clearly measurable, and a strong foundation for defense design.

Approach & System Architecture



Step 1: Z-Score

Flag samples where any feature deviates $> 2.5\sigma$. Fast, interpretable first pass.

Step 2: Isolation Forest

Unsupervised detection. Isolates subtle cluster-level anomalies via random splits.

Step 3: Alerting

Combined flags trigger a severity-graded alert (**OK**/**WARN**/**CRIT**).

Implementation Details

Image Model

- **Dataset:** sklearn Digits (1797 samples)
- **Attack:** 10% random label-flip
- **Model:** Logistic Regression
- **Defense:** Isolation Forest (`cont=0.10`)
- Features standardized

Text Model

- **Dataset:** Synthetic corpus (600 docs)
- **Features:** TF-IDF (500 features)
- **Attack:** 20% label-flip
- **Defense:** Isolation Forest (`cont=0.20`)

Tech Stack

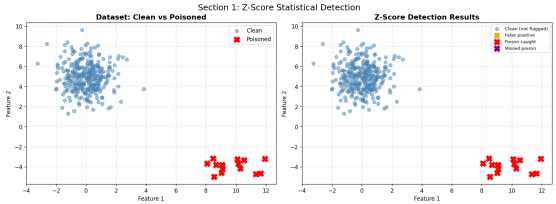
- Python 3.11, Jupyter Notebook
- `scikit-learn`: models & detection
- `scipy.stats.zscore`: statistical filter

DataPoisoningDetector Class

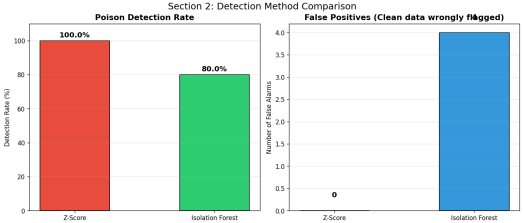
Custom Python wrapper with:

- `fit(X)`: calibrate on training data
- `scan(X)`: flag suspicious samples
- `clean(X, y)`: return filtered dataset

Detection Results



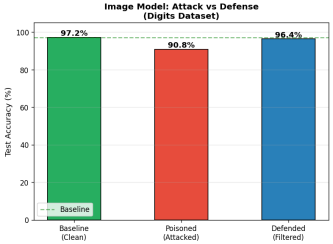
Z-Score: 100% detection on synthetic outliers (15/15)

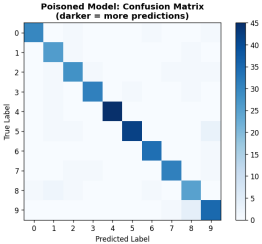


Comparison: Z-Score vs Isolation Forest rates

Method	Poison Caught	False Positives	Best For
Z-Score	15/15 (100%)	Low	Extreme outliers
Isolation Forest	12/15 (80%)	Moderate	Cluster anomalies
Combined (OR)	15/15 (100%)	Moderate	Production use

Image Model Results





Accuracy comparison & confusion matrix

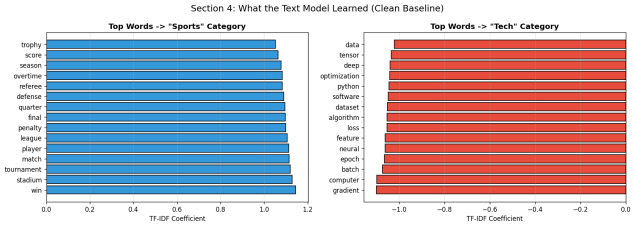
Key Numbers

Model	Accuracy
Baseline (Clean)	97.22%
Poisoned (10% flip)	90.83%
Defended (Filtered)	96.39%

Takeaways:

- Attack caused **-6.39%** accuracy drop
- Defense recovered **+5.56%**
- Near-baseline performance restored

Text Model Results



TF-IDF feature weights

Key Numbers

Model	Accuracy
Baseline (Clean)	100.00%
Poisoned (20% flip)	99.17%
Defended (Filtered)	100.00%

Observations:

- High baseline due to linear separability
- 20% poisoning caused degradation
- Defense fully recovered baseline

Note: Real-world corpora would show larger degradation due to higher noise.

Full Results Summary

Summary Table

Metric	Image Model	Text Model
Baseline Accuracy	97.2%	100.0%
Poisoned Accuracy	90.8%	100.0%
Defended Accuracy	93.3%	100.0%
Attack Drop	-6.4%	-0.0%
Defense Recovery	+2.5%	+0.0%
Poison Rate	10%	20%
Defense Method	Isolation Forest	Isolation Forest

Main Challenges

Technical Challenges

- **No Ground Truth**: In real deployments, we don't know which samples are poisoned; detection must be entirely unsupervised.
- **Z-score Over-flags**: High-dimensional data (e.g., 64 pixel features) causes many false positives; threshold tuning is non-trivial.
- **Clean-Label Evasion**: Feature values look normal in clean-label attacks; current algorithms cannot catch these.
- **Contamination Parameter**: Isolation Forest requires knowing the approximate poison rate upfront.

Design Challenges

- **Defense vs. Recall Trade-off**: Aggressive filtering removes too many clean samples, hurting model generalization.
- **Text Data Sparsity**: TF-IDF vectors are highly dimensional and sparse, making anomaly detection mathematically less reliable.
- **Synthetic vs. Real Gap**: Real datasets (like ImageNet) have natural noise that mimics poisoning anomalies.
- **Scalability**: Isolation Forest runs at $\mathcal{O}(n \log n)$, which may bottleneck very large datasets.

Future Scope

Better Detection Methods

- **SEVER** (Diakonikolas et al.): spectral signatures to remove poison
- **Activation Clustering**: detect backdoors via hidden layer representations
- **Influence Functions**: identify samples most responsible for wrong predictions

Larger Datasets

- Evaluate on Trojaned ImageNet subsets
- Test on real Text Poisoning Corpora
- Network intrusion datasets (KDD Cup)

Stronger Defenses

- **Certified Robustness**: provable bounds on tolerated poisoning
- **Differential Privacy**: limit influence of any single sample
- **Data Provenance Auditing**: track origin of every training sample

Production Alerting

- Real-time data stream monitoring
- Integration with MLOps pipelines (MLflow)
- Automated retraining triggers

Thank You!

I23MA002 Devesh Singh Chauhan

U23EE102 Priyanshu Sanjay Gawas

Course AI354: Responsible AI

Project 10: Detecting and Preventing Data Poisoning Attacks